

Source: Google Cloud Documentation, *MLOps: Continuous Delivery and Automation Pipelines in Machine Learning*

Section: MLOps Level 0: Manual Process

Licence: Creative Commons Attribution 4.0 International (CC BY 4.0)

Original source: Google Cloud Architecture Center

Content reproduced from a larger document and provided for educational use.

Contributors:

Authors:

Jarek Kazmierczak | Solutions Architect

Khalid Salama | Staff Software Engineer, Machine Learning

Valentín Huerta | AI Engineer

Other contributor:

Sunil Kumar Jang Bahadur | Customer Engineer

MLOps: Continuous delivery and automation pipelines in machine learning

Last reviewed 2024-08-28 UTC

This document discusses techniques for implementing and automating continuous integration (CI), continuous delivery (CD), and continuous training (CT) for machine learning (ML) systems. This document applies primarily to predictive AI systems.

Data science and ML are becoming core capabilities for solving complex real-world problems, transforming industries, and delivering value in all domains. Currently, the ingredients for applying effective ML are available to you:

- Large datasets
- Inexpensive on-demand compute resources
- Specialized accelerators for ML on various cloud platforms

Source: Google Cloud Documentation (CC BY 4.0). Extracted from *MLOps: Continuous Delivery and Automation Pipelines in Machine Learning*.

- Rapid advances in different ML research fields (such as computer vision, natural language understanding, generative AI, and recommendations AI systems).

Therefore, many businesses are investing in their data science teams and ML capabilities to develop predictive models that can deliver business value to their users.

This document is for data scientists and ML engineers who want to apply [DevOps](#) principles to ML systems (MLOps). *MLOps* is an ML engineering culture and practice that aims at unifying ML system development (Dev) and ML system operation (Ops). Practicing MLOps means that you advocate for automation and monitoring at all steps of ML system construction, including integration, testing, releasing, deployment and infrastructure management.

Data scientists can implement and train an ML model with predictive performance on an offline holdout dataset, given relevant training data for their use case. However, the real challenge isn't building an ML model, the challenge is building an integrated ML system and to continuously operate it in production. With the long history of production ML services at Google, we've learned that there can be many pitfalls in operating ML-based systems in production. Some of these pitfalls are summarized in [Machine Learning: The High Interest Credit Card of Technical Debt](#).

As shown in the following diagram, only a small fraction of a real-world ML system is composed of the ML code. The required surrounding elements are vast and complex.

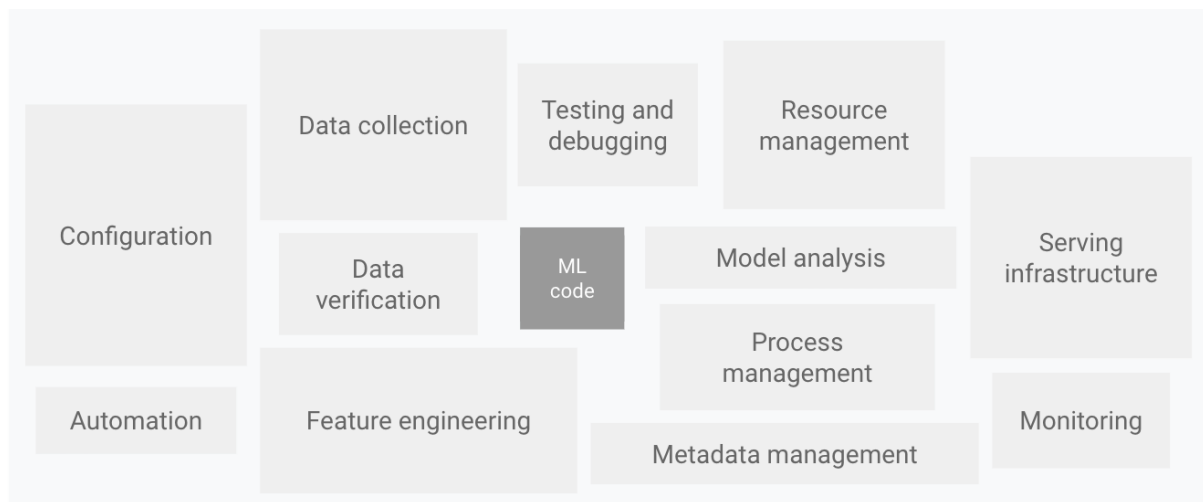


Figure 1. Elements for ML systems. Adapted from [Hidden Technical Debt in Machine Learning Systems](#).

The preceding diagram displays the following system components:

- Configuration
- Automation
- Data collection
- Data verification
- Testing and debugging
- Resource management
- Model analysis
- Process and metadata management
- Serving infrastructure
- Monitoring

To develop and operate complex systems like these, you can apply DevOps principles to ML systems (MLOps). This document covers concepts to consider when setting up an MLOps environment for your data science practices, such as CI, CD, and CT in ML.

The following topics are discussed:

- DevOps versus MLOps
- Steps for developing ML models
- MLOps maturity levels
- MLOps for generative AI

DevOps versus MLOps

[DevOps](#) is a popular practice in developing and operating large-scale software systems.

This practice provides benefits such as shortening the development cycles, increasing deployment velocity, and dependable releases. To achieve these benefits, you introduce two concepts in the software system development:

- [Continuous integration \(CI\)](#)
- [Continuous delivery \(CD\)](#)

An ML system is a software system, so similar practices apply to help guarantee that you can reliably build and operate ML systems at scale.

However, ML systems differ from other software systems in the following ways:

- Team skills: In an ML project, the team usually includes data scientists or ML researchers, who focus on exploratory data analysis, model development, and

Source: Google Cloud Documentation (CC BY 4.0). Extracted from *MLOps: Continuous Delivery and Automation Pipelines in Machine Learning*.

experimentation. These members might not be experienced software engineers who can build production-class services.

- **Development:** ML is experimental in nature. You should try different features, algorithms, modeling techniques, and parameter configurations to find what works best for the problem as quickly as possible. The challenge is tracking what worked and what didn't, and maintaining reproducibility while maximizing code reusability.
- **Testing:** Testing an ML system is more involved than testing other software systems. In addition to typical unit and integration tests, you need data validation, trained model quality evaluation, and model validation.
- **Deployment:** In ML systems, deployment isn't as simple as deploying an offline-trained ML model as a prediction service. ML systems can require you to deploy a multi-step pipeline to automatically retrain and deploy models. This pipeline adds complexity and requires you to automate steps that are manually done before deployment by data scientists to train and validate new models.
- **Production:** ML models can have reduced performance not only due to suboptimal coding, but also due to constantly evolving data profiles. In other words, models can decay in more ways than conventional software systems, and you need to consider this degradation. Therefore, you need to track summary statistics of your data and monitor the online performance of your model to send notifications or roll back when values deviate from your expectations.

ML and other software systems are similar in continuous integration of source control, unit testing, integration testing, and continuous delivery of the software module or the package. However, in ML, there are a few notable differences:

- CI is no longer only about testing and validating code and components, but also testing and validating data, data schemas, and models.
- CD is no longer about a single software package or a service, but a system (an ML training pipeline) that should automatically deploy another service (model prediction service).
- CT is a new property, unique to ML systems, that's concerned with automatically retraining and serving the models.

The following section discusses the typical steps for training and evaluating an ML model to serve as a prediction service.

Data science steps for ML

In any ML project, after you define the business use case and establish the success criteria, the process of delivering an ML model to production involves the following steps. These steps can be completed manually or can be completed by an automatic pipeline.

1. Data extraction: You select and integrate the relevant data from various data sources for the ML task.
2. Data analysis: You perform [exploratory data analysis \(EDA\)](#) to understand the available data for building the ML model. This process leads to the following:

Source: Google Cloud Documentation (CC BY 4.0). Extracted from *MLOps: Continuous Delivery and Automation Pipelines in Machine Learning*.

- a. Understanding the data schema and characteristics that are expected by the model.
 - b. Identifying the data preparation and feature engineering that are needed for the model.
3. Data preparation: The data is prepared for the ML task. This preparation involves data cleaning, where you split the data into training, validation, and test sets. You also apply data transformations and feature engineering to the model that solves the target task. The output of this step are the *data splits* in the prepared format.
4. Model training: The data scientist implements different algorithms with the prepared data to train various ML models. In addition, you subject the implemented algorithms to hyperparameter tuning to get the best performing ML model. The output of this step is a trained model.
5. Model evaluation: The model is evaluated on a [holdout test set](#) to evaluate the model quality. The output of this step is a set of metrics to assess the quality of the model.
6. Model validation: The model is confirmed to be adequate for deployment—that its predictive performance is better than a certain baseline.
7. Model serving: The validated model is deployed to a target environment to serve predictions. This deployment can be one of the following:
 - a. Microservices with a REST API to serve online predictions.
 - b. An embedded model to an edge or mobile device.
 - c. Part of a batch prediction system.

8. Model monitoring: The model predictive performance is monitored to potentially invoke a new iteration in the ML process.

The level of automation of these steps defines the *maturity* of the ML process, which reflects the velocity of training new models given new data or training new models given new implementations. The following sections describe three levels of MLOps, starting from the most common level, which involves no automation, up to automating both ML and CI/CD pipelines.

MLOps level 0: Manual process

Many teams have data scientists and ML researchers who can build state-of-the-art models, but their process for building and deploying ML models is entirely manual. This is considered the *basic* level of maturity, or level 0. The following diagram shows the workflow of this process.

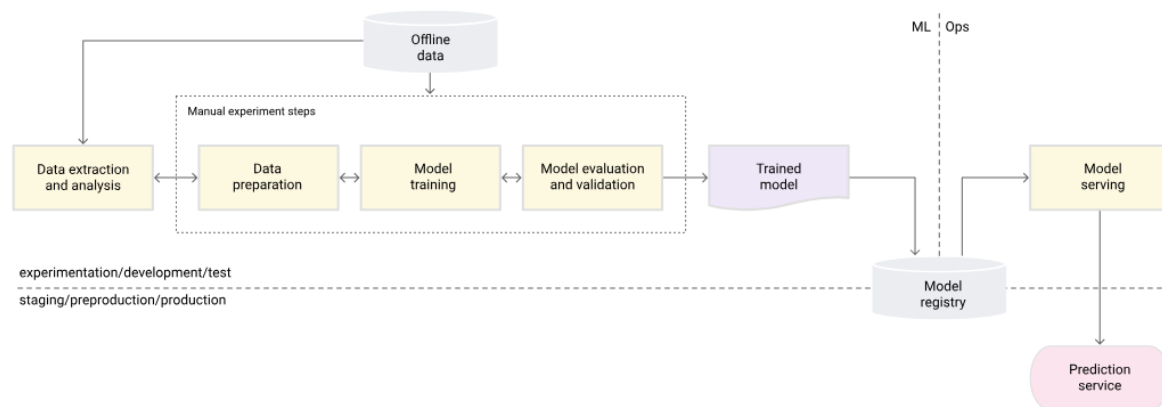


Figure 2. Manual ML steps to serve the model as a prediction service.

Characteristics

The following list highlights the characteristics of the MLOps level 0 process, as shown in Figure 2:

- Manual, script-driven, and interactive process: Every step is manual, including data analysis, data preparation, model training, and validation. It requires

Source: Google Cloud Documentation (CC BY 4.0). Extracted from *MLOps: Continuous Delivery and Automation Pipelines in Machine Learning*.

manual execution of each step, and manual transition from one step to another. This process is usually driven by experimental code that is interactively written and executed in notebooks by data scientists, until a workable model is produced.

- **Disconnection between ML and operations:** The process separates data scientists who create the model and engineers who serve the model as a prediction service. The data scientists hand over a trained model as an artifact to the engineering team to deploy on their API infrastructure. This handoff can include putting the trained model in a storage location, checking the model object into a code repository, or uploading it to a models registry. Then engineers who deploy the model need to make the required features available in production for low-latency serving, which can lead to [training-serving skew](#).
- **Infrequent release iterations:** The process assumes that your data science team manages a few models that don't change frequently—either changing model implementation or retraining the model with new data. A new model version is deployed only a couple of times per year.
- **No CI:** Because few implementation changes are assumed, CI is ignored. Usually, testing the code is part of the notebooks or script execution. The scripts and notebooks that implement the experiment steps are source controlled, and they produce artifacts such as trained models, evaluation metrics, and visualizations.
- **No CD:** Because there aren't frequent model version deployments, CD isn't considered.
- **Deployment refers to the prediction service:** The process is concerned only with deploying the trained model as a prediction service (for example, a microservice with a REST API), rather than deploying the entire ML system.
- **Lack of active performance monitoring:** The process doesn't track or log the model predictions and actions, which are required in order to detect model performance degradation and other model behavioral drifts.

The engineering team might have their own complex setup for API configuration, testing, and deployment, including security, regression, and load and canary testing. In addition, production deployment of a new version of an ML model usually goes through A/B testing or online experiments before the model is promoted to serve all the prediction request traffic.

Challenges

MLOps level 0 is common in many businesses that are beginning to apply ML to their use cases. This manual, data-scientist-driven process might be sufficient when models are rarely changed or trained. In practice, models often break when they are deployed in the real world. The models fail to adapt to changes in the dynamics of the environment, or changes in the data that describes the environment. For more information, see [Why Machine Learning Models Crash and Burn in Production](#).

To address these challenges and to maintain your model's accuracy in production, you need to do the following:

- Actively monitor the quality of your model in production: Monitoring lets you detect performance degradation and model staleness. It acts as a cue to a new experimentation iteration and (manual) retraining of the model on new data.
- Frequently retrain your production models: To capture the evolving and emerging patterns, you need to retrain your model with the most recent data. For example, if your app recommends fashion products using ML, its recommendations should adapt to the latest trends and products.
- Continuously experiment with new implementations to produce the model: To harness the latest ideas and advances in technology, you need to try out new implementations such as feature engineering, model architecture, and hyperparameters. For example, if you use computer vision in face detection, face patterns are fixed, but better new techniques can improve the detection accuracy.

To address the challenges of this manual process, MLOps practices for CI/CD and CT are helpful. By deploying an ML training pipeline, you can enable CT, and you can set up a CI/CD system to rapidly test, build, and deploy new implementations of the ML pipeline. These features are discussed in more detail in the next sections.